

→ **Upsides:** no need to transfer entire cache block into cache (write only invalidates and updates a subblock)
more freedom in transferring subblocks into the cache (cache block does not fully be in cache, how many subblocks do you transfer on a read?)

Downsides: more design complexity (more valid and dirty bits), may not fully exploit spatial locality

→ **Instruction vs. Data Caches:** Separate or Unified?

→ **Upsides for Unified:** Dynamic sharing of cache space → better overall cache utilization → no overprovisioning that can happen with partitioned cache space

Downsides: Ins./data can evict/through each other → no guaranteed space for either

Ins. and data are accessed in different stages of pipeline → where do we place unified cache for fast access?

→ First level caches are almost always split → mainly for last reason, other level caches almost always unified

Multi-level cache design:

First-level caches (ins. and data):

→ decisions affected by cycle time and pipeline structure, small with low associativity and latency, tag and data store usually accessed in parallel

Second-level caches: decisions balance hit rate and access latency, usually large and highly associative, latency less important, tag store and data store can be accessed serially

Further-level (larger) caches: Access energy is a larger problem due to cache sizes, tag/data store usually accessed serially

Serial vs. parallel access of cache levels:

→ **Parallel:** next level cache accessed in parallel with previous level → speculative

→ faster access to data if cache misses, unnecessary access if it hits

→ **Serial:** next level accessed only for cache misses

→ slower access for misses, but no wasted accesses to next level if previous hits

→ next level doesn't have same accesses as previous → previous acts as filter for some temporal/spatial locality

→ **Hierarchies:**

Inclusive: block in inner level is always included in outer levels → simplifies cache coherence

Exclusive: block in inner level does not exist in outer level → better use of space in entire hierarchy

Non-inclusive: block in inner level may or may not be included in outer level → relaxes design decisions, used in many Intel processors

→ back to improving cache performance → reduce miss rate, reduce miss latency, reduce hit latency

→ **Software approaches** to improve hit rate:

→ Restructuring data access patterns → change data layout → example: matrix sum, column vs. row major

Example: If column-major
□ $x[i+1,j]$ follows $x[i,j]$ in memory
□ $x[i,j+1]$ is far away from $x[i,j]$

Poor code
for i = 1, rows
for j = 1, columns
sum = sum + x[i,j]

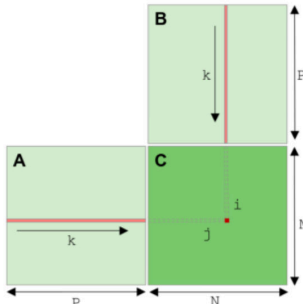
Better code
for j = 1, columns
for i = 1, rows
sum = sum + x[i,j]

→ This is called loop interchange

Blocking: divide loops operating on arrays into computation chunks so each chunk can hold its data in the cache → avoids cache conflicts between different chunks → divide the working set s.t. each piece fits in cache (Tiling)

Naïve Matrix Multiplication (I)

- Matrix multiplication: $C = A \times B$
- Consider two input matrices A and B in row-major layout
- A size is $M \times P$
- B size is $P \times N$
- C size is $M \times N$



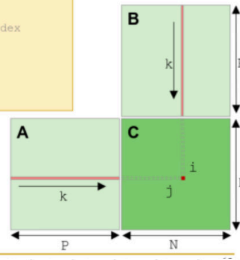
Naïve Matrix Multiplication (II)

- Naïve implementation of matrix multiplication has poor cache locality

```
#define A(i,j) matrix_A[i * P + j]
#define B(i,j) matrix_B[i * N + j]
#define C(i,j) matrix_C[i * N + j]

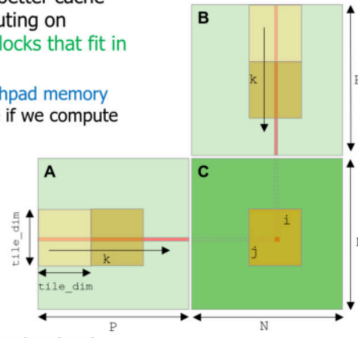
for (i = 0; i < M; i++) { // i = row index
  for (j = 0; j < N; j++) { // j = column index
    C(i, j) = 0; // Set to zero
    for (k = 0; k < P; k++) // Row x Col
      C(i, j) += A(i, k) * B(k, j);
  }
}
```

Consecutive accesses to B are far from each other, in different cache lines. Every access to B is likely to cause a cache miss



Tiled Matrix Multiplication (I)

- We can achieve better cache locality by computing on smaller tiles or blocks that fit in the cache
- Or in the scratchpad memory and register file if we compute on a GPU



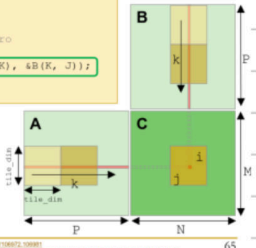
Tiled Matrix Multiplication (II)

- Tiled implementation operates on submatrices (tiles or blocks) that fit fast memories (cache, scratchpad, RF)

```
#define A(i,j) matrix_A[i * P + j]
#define B(i,j) matrix_B[i * N + j]
#define C(i,j) matrix_C[i * N + j]

for (I = 0; I < M; I += tile_dim) {
  for (J = 0; J < N; J += tile_dim) {
    Set_to_zero(C(I, J)); // Set to zero
    for (K = 0; K < P; K += tile_dim) {
      Multiply_tiles(&C(I, J), &A(I, K), &B(K, J));
    }
  }
}
```

Multiply small submatrices (tiles or blocks) of size tile_dim x tile_dim



Restructuring Data Layout

```
struct Node {
  struct Node* next; // Frequently accessed
  int key; // Frequently accessed
  char [256] name; // Rarely accessed
  char [256] school; // Rarely accessed
}

while (node) {
  if (node->key == input-key) {
    // access other fields of node
    node = node->next; // Frequently accessed
  }
}
```

→ Restructuring Data Layout: Assume pointer-based traversal (e.g. linked list)

→ for a large linked list with unique keys, the hit rate is poor because rarely accessed fields occupy most of cache blocks

→ Idea: separate rarely-accessed fields of data structure and pack them into separate data structure

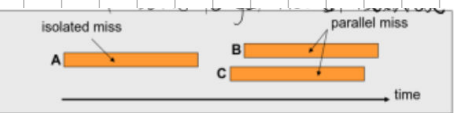
```
struct Node {
  struct Node* next;
  int key;
  struct Node-data* node-data;
}

struct Node-data {
  char [256] name;
  char [256] school;
}
```

→ General idea: separate frequently-accessed data from rarely-accessed data so they're not in same cache block

Miss cost: what does it depend on?

- where is the miss serviced from?
- what level of cache in the hierarchy, DRAM row hit vs. row conflict, queuing delays in mem. controller and interconnect, local vs. remote memory
- how much does the miss stall the processor?
- is it overlapped with other latencies? Is the data immediately needed by the processor?...

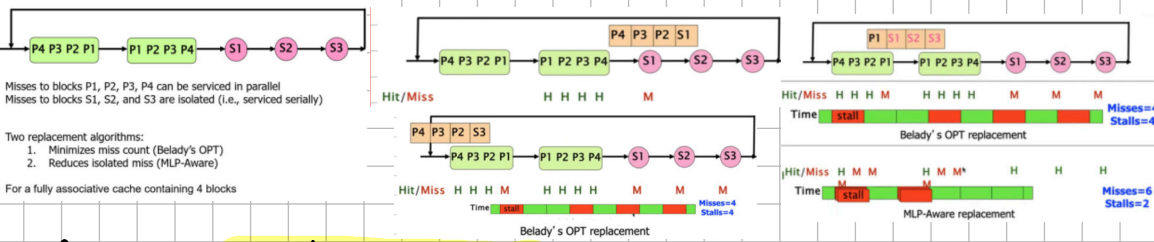


→ Memory Level Parallelism: generate and service multiple memory accesses in parallel

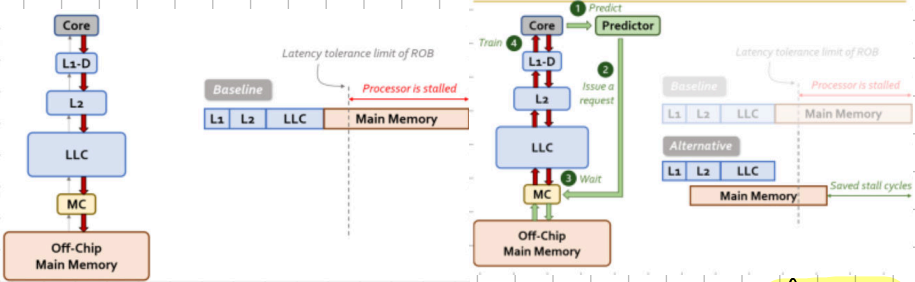
→ OoO greatly improved MLP, but it varies as some misses are parallel and others isolated

→ you would assume reducing misscount would reduce memory⁷⁴-level stall time, which is what traditional cache management policies assume, but misses with varying MLP break this

→ Eliminating an isolated miss helps performance more than eliminating a parallel miss, eliminating a higher-latency miss helps performance more than eliminating a lower-latency miss



→ Predictors for if a load/store is cached allow us to access main memory directly if it's unlikely, saving time vs. accessing serially



Victim cache: small, fully associative buffer to store recently evicted blocks
 upside:
 → can avoid ping-ponging of blocks mapped to same set (if 2 cache blocks continuously accessed in nearby time conflict with each other)

Key Data Patterns in Real Applications

- Zero Values:** initialization, sparse matrices, NULL pointers
 0x00000000 0x00000000 0x00000000 0x00000000 ...
- Repeated Values:** common initial values, adjacent pixels
 0x000000FF 0x000000FF 0x000000FF 0x000000FF ...
- Narrow Values:** small values stored in a big data type
 0x00000000 0x0000000B 0x00000003 0x00000004 ...
- Other Patterns:** pointers to the same memory region
 0xC04039C0 0xC04039C8 0xC04039D0 0xC04039D8 ...

downsides: increases miss latency if accessed serially with L2, adds complexity

→ because of these data comments being so common, there are **cache compression algorithms** (surely not exam relevant?) that allow us to store many more pieces of data in a cache of the same size

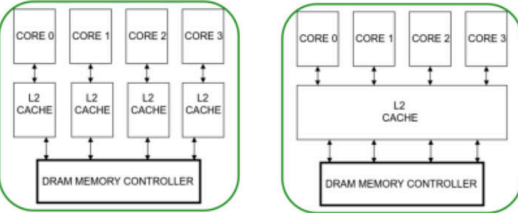
Resource Sharing:

→ Allowing multiple hardware contexts to use a HW resource → example: pipelines, caches, memory...

→ **Upsides:** improves utilization/efficiency → throughput → when resource left idle by thread, another thread can use it
 reduces communication latency → example: **shared cache** for multiple CPU cores compatible with shared memory programming model

→ **Downsides:** Resource sharing causes contention, when resource not idle, another thread cannot use it
 Can reduce each or some threads' performance
 Eliminates performance isolation → inconsistent perf. across runs
 Uncontrolled sharing causes unfairness, starvation

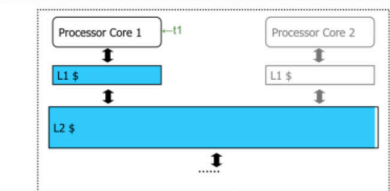
Private cache: Cache belongs to one core (shared block can be in multiple caches), **Shared cache:** multiple cores use cache



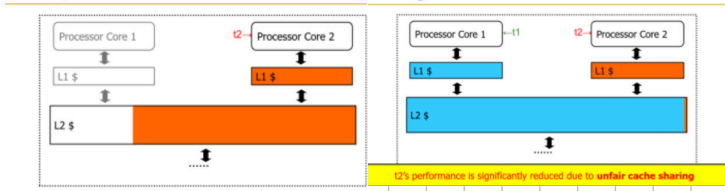
→ **Upsides to sharing:** High effective capacity, dynamic partitioning of available space (no fragmentation from static partitioning, if one core doesn't utilize space, another can), easier to maintain coherence (cache block is in single location)

→ **Downsides:** Slower access (cache not coupled to core), cores cause conflict misses due to other cores' accesses (inter-core interference, some cores destroy hit rate of others), guaranteeing minimum level of service/fairness to each core is harder (space, bandwidth)

Example: One Problem with Shared Caches



Example: One Problem with Shared Caches



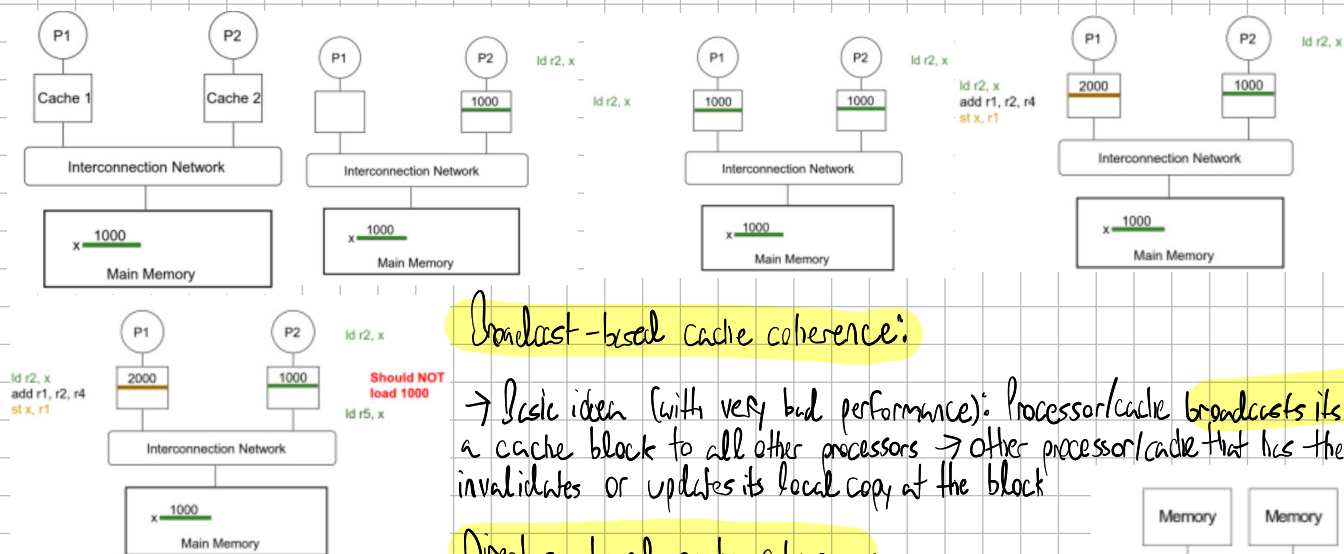
→ **Balancing Sharing and Partitioning:** Design shared resources s.t. they are efficiently utilized, controllable, partitionable

Shared Memory Model: As you know from PProg, threads in parallel programs communicate through shared memory

Processor 0 Processor 1
Mem[A] = 1 ...
.....
 Print Mem[A]

→ each read needs latest write from any processor → synchronization between threads
→ what if Mem[A] is cached?

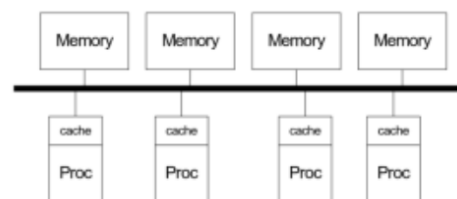
Cache coherence: ensuring consistent state across caches of different processors



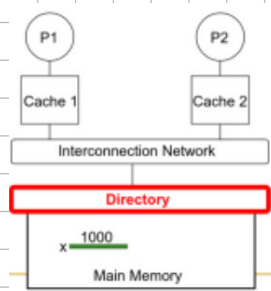
Broadcast-based cache coherence:

→ Basic idea (with very bad performance): Processor/cache broadcasts its write/update to a cache block to all other processors → other processor/cache that has the block either invalidates or updates its local copy at the block

Directory-based cache coherence:



→ Idea: central directory tracks which caches contain every possible cache block, every cache consults this for coherence



Prefetching:

Idea: fetch data before it's needed by program

→ Why? Mem. latency is high, accurate and early prefetching can eliminate that → can eliminate compulsory code misses

→ we need to predict which address is needed in the future → works for predictable miss address patterns

→ Misprediction in prefetching doesn't affect correctness → wrongly prefetched data is not used, no need for state recovery

→ Modern systems prefetch at cache block granularity

→ Prefetching can reduce both miss rate and miss latency, can be done by HW, compiler, programmer

What addresses to prefetch:

→ Prefetching useless data wastes resources → memory bandwidth/cache space/energy

→ Accurate prediction is important → Prefetch accuracy = $\frac{\text{used prefetches}}{\text{sent prefetches}}$

→ Prefetching algorithm determines what to prefetch → past access patterns and software knowledge of data structures

Some Predictable Address Access Patterns?

When to initiate a prefetch request:

→ Too early → prefetched data may not be used before eviction from storage

→ Too late → may not hide all of memory latency (mem. load already in progress)

→ Can be made more timely: more aggressive → stay far ahead of processor's demand access stream (HW), moving prefetch insns earlier in the code (SW)

$$A, A+1, A+2, A+3, A+4, \dots$$
$$B, B+42, B+84, B+126, \dots$$

C, C+2, C+5, C+9, C+11, C+14, C+18, C+20, C+23, C+27, ...

X, Y, T, O, R, S, X, Y, T, A, B, C, D, E, X, Y, T, F, G, H, X, Y, T, ...

A+1, A+67, A+18, A+7, A+99, Z+1, Z+67, Z+18, Z+7, Z+99,
P+1, P+67, P+18, P+7, P+99, ...

Where to prefetch to:

→ Which level? Memory to L1 or lower levels? between cache levels?

→ Where in the cache? Do we treat prefetched blocks the same as demand-fetched blocks? Prefetched blocks are not known to be needed

→ Should replacement policy favor demand-fetched blocks?

→ Where should the HW prefetcher be in the mem. hierarchy? → What access patterns does it see? L1 hits and misses, L1 misses only, L2 misses only?

→ Giving more details on access pattern: better accuracy and coverage in prefetching, but prefetcher has to examine more requests (bandwidth intensive, more ports into prefetcher)

How to prefetch:

→ SW prefetching: ISA provides prefetch ins., Programmer/Compiler inserts prefetch ins. into code, usually only works well for "regular access patterns"

→ HW prefetching: Specialized HW monitors memory access, memorizes/finds/learns address strided/patterns/correlations, generates prefetch addresses

→ Execo.-based prefetching: "thread" is executed to prefetch data for main program, can be generated by SW or HW

Performance metrics (exam relevant):

Accuracy: used prefetches / sent prefetches, Coverage: prefetched misses / all misses, Timeliness: on-time prefetches / all prefetches

→ Bandwidth consumption: mem. bandwidth consumed with prefetcher/without prefetcher → good news: can utilize bus bandwidth if available

→ Cache pollution: extra demand misses due to prefetch placement in cache, harder to quantify

Stride prefetchers:

■ Consider the following strided memory access pattern:

- $A, A+N, A+2N, A+3N, A+4N, \dots$
- Stride = N

→ Idea: record stride between consecutive mem. accesses, if stable, use it to predict next M mem. accesses

2 types:

Inso.-based stride prefetching

Instruction-Based Stride Prefetching

Load Inst PC	Load Inst. PC (tag)	Last Address Referenced	Last Stride	Confidence
				

→ each load/store can lead to mem. access pattern with diff. stride → can detect strides caused by each ins.

→ Timeliness can be an issue: initiating prefetch when load is fetched next time may be too late → potential solution: look ahead in ins. stream

Memory-Region-Based Stride Prefetching

Cache Block Address	Address tag	Stride	Control/Confidence
			

Mem. region-based stride prefetching:

→ Can detect strided mem. access patterns that appear due to multiple inso → $A, A+N, A+2N, \dots$

→ Stream prefetching (stream buffers) is a special case of mem. region based stride prefetching where $N=1$

→ Stride prefetchers do well on simple regular patterns, there are also complex regular patterns (e.g. multiple regular strides like $+1, +2, +3, +1, +2, \dots$)

and irregular patterns (linked data structure traversals, random accesses, concurrent data structure access...) which require more complex, non-exam relevant prefetchers

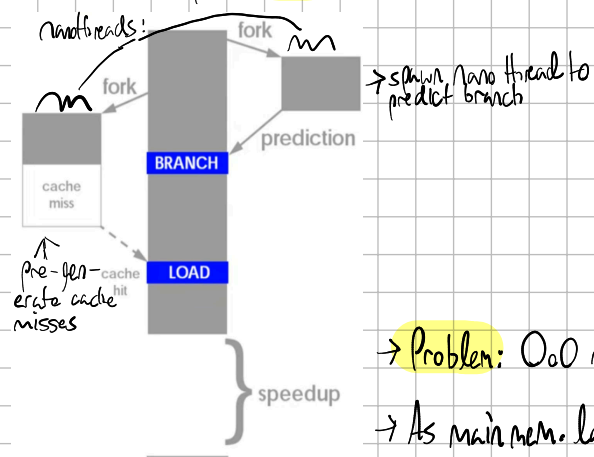
Execution-Based Prefetching:

Idea: pre-execute piece of program solely for prefetching data → only need to pre-execute code that leads to cache misses

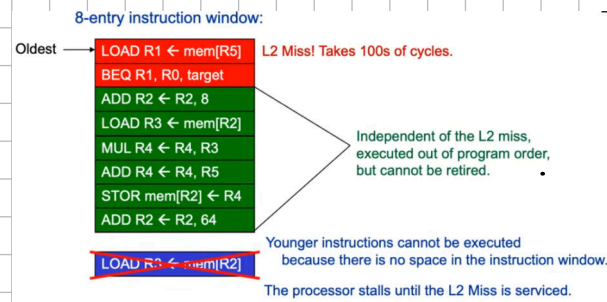
→ **Speculative thread**: Pre-executed program piece can be considered a "thread"

→ Speculative thread can be executed on a separate processor/core, on separate HW thread context (F/GMT), on same thread context in idle cycles (during cache misses)

Thread-based pre-execution:

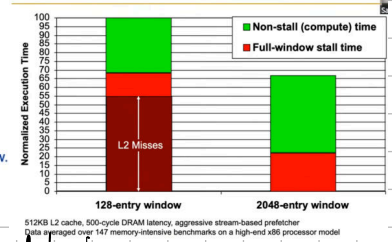


Runahead Execution:



→ long-latency cache misses are responsible for most full-window stalls

Impact of Long-Latency Cache Misses



→ Problem: OoO requires large ins. windows to tolerate modern main mem. latencies

→ As main mem. latency increases, ins. window size should also increase to tolerate said latency

→ large ins. window challenging if we also want low energy consumption, short cycle time, low design/verification complexity

→ **Runahead Execution**: meant to obtain memory-level parallelism benefits of large ins. window

→ when oldest ins. is long-latency cache miss: checkpoint arch. state, enter runahead mode

→ in runahead mode: speculatively pre-execute ins. (with minimal stalling), purpose of this is to generate prefetches

→ long-latency ins. are marked INV and dropped → enables room for later ins. in window

→ when original miss returns: restore checkpoint, flush pipeline, resume normal execution

Benefits: Instead of stalling during L2 cache miss: processor speculatively executes the program far into ins. stream without stalling for long-latency ins.

→ pre-executed loads/stores independent of L2-miss ins. generate very accurate data prefetches for both regular/irregular data prefetches

→ ins. on predicted program path prefetched into ins. cache and outer cache levels

→ HW prefetcher and branch predictor trained using future access information

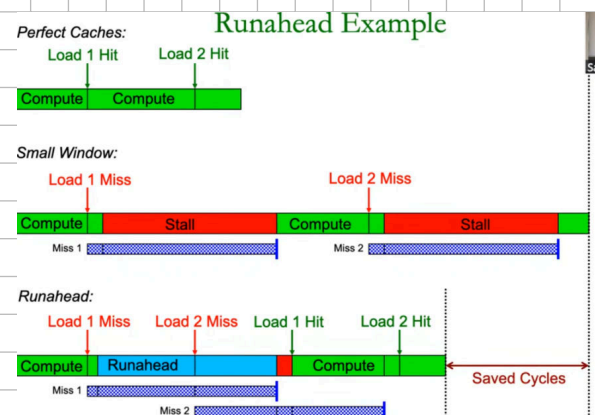
→ Simple to implement: most needed HW built in already, no need to construct special-purpose pre-execution thread for prefetching

→ No waste of HW context → prefetching using main thread context

Disadvantages: extra executed ins., limited by branch prediction accuracy, cannot prefetch dependent cache misses

→ effectiveness limited by available memory-level parallelism

→ Prefetch distance (how far ahead) limited by memory latency



Ok just a bunch of space I didn't plan for

Virtual Memory: (VMem)

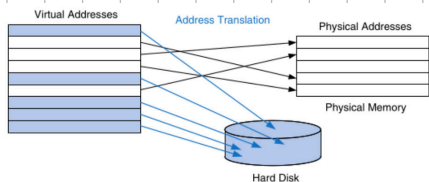
→ Hard drives are even larger and slower than DRAM → we can give the programmer the illusion of a much larger memory with DRAM speeds

→ Example: with 128MB DRAM and 4GB HDD can effectively provide 2GB of mem. using the HDD → virtual memory, the smaller DRAM is physical memory (we will call it that here)

→ Program can access data anywhere in VMem → must use virtual addresses that specify location in VMem

→ physical mem. holds a subset of most recently accessed VMem → phys. mem. acts as **cache** for VMem

→ **Different terminology** for core caching principles from previous section: cache block → virtual page (usually 4kb in size), physical mem. is also divided into physical pages of the same size → virtual page may be located in phys. mem. or on the disk



→ in the image, rectangles are pages, we see this split

→ **address translation**: determining physical address from virtual address

→ **page fault** (≡ cache miss): processor attempts to access virtual addr. not in phys. mem., OS loads page into phys. mem. from hard disk

→ avoiding page faults caused by conflicts → any virtual page can map to any physical page → phys. mem. behaves as **fully associative cache** for VMem

→ conventional fully associative cache: every cache block has comparator checking most significant address bits against tag to determine if cache hits

→ **analogous VMem system**: each phys. page would need comparator to check address bits against tags to see if virtual page maps to physical page → this would be excessively expensive → **page table is used for address translation**

→ page table contains entry for each virtual page, indicating its location in phys. mem. or that it is on the disk

→ each load/store ins. requires **page table access** followed by **phys. mem. access**

→ page table access **translates virtual addr.** used by program to physical address, which is used to **read/write data**

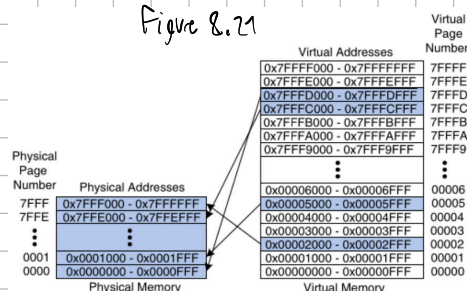
→ page table is so large it is located in phys. mem. → each load/store requires 2 phys. mem. accesses

→ **translation lookaside buffer (TLB)** speeds up addr. translation by caching most commonly used page table entries

→ more **cache/VMem "isomorphisms"**: block size ≡ page size, block offset ≡ page offset, tag ≡ virtual page number

Address Translation:

Figure 8.21



→ here, we have **2GB VMem, 128 MB phys. mem.**, all in **4KB pages** using 32-bit addresses (MIPS)

→ $2GB = 2^{31}$ byte VMem, only 31 address LSB's used, last always 0

→ **page size**: $4KB = 2^{12}$ pages → $2^{31}/2^{12} = 2^{19}$ virtual pages, $2^{27}/2^{12} = 2^{15}$ physical pages ($128MB = 2^{27}$ bytes)

→ virtual page number → 19 bits, physical → 15 bits → phys. mem. can only hold **1/16** th of virtual pages at any given time, rest on disk

→ virtual page 5 maps to physical page 1, virtual page 0x7FFFC maps to phys. page 0x7FFE, ...

→ virtual addr. 0x53FB (offset of 0x3FB in virtual page 5) would map to phys. address 0x13FB (offset of 0x3FB within phys. page 1)

→ 12 LSB's of virtual/physical mem. addr. are identical (0x3FB) → **page offset** within virtual and physical pages → only page number needs to be translated to obtain physical addr. from virtual addr.

→ 12 LSB's indicate **page offset**, require no translation

→ upper 19 bits of virt. addr. specify virtual page number (VPN), are translated into 15-bit physical page number (PPN)

Page Table:

Physical Page Number	Virtual Page Number
0	7FFF
0	7FFE
1	0x0000
1	0x7FFE
0	7FFC
0	7FFB
0	7FFA
...	...
0	0007
0	0006
1	0005
0	0004
0	0003
1	0x7FFF
0	0002
0	0001
0	0000

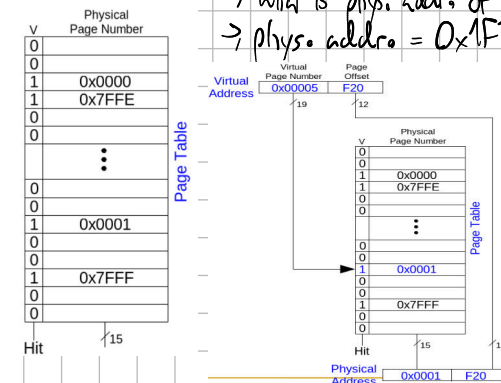
Figure 8.23 The page table for Figure 8.21

- processor uses it to translate virtual to physical addresses, page table has entry for each virtual page
- each entry contains physical page number and valid bit → if valid bit = 1, virtual page maps to physical page in its entry, else it's on the disk
- stored in phys. mem. because of its size → assume it is stored as a contiguous array
- ← example page table is indexed with VPN → example: entry 5 shows virtual page 5 maps to physical page 1, entry 6 invalid → virt. page 6 on disk

- page table can be stored anywhere in phys. mem. → OS decides
- Processor uses dedicated page table register (PTBR) to store base addr. of page table in phys. mem.
- perform load/store: processor must translate virtual address to physical address, then access data at that physical address
- Physical address = $VPN * \text{page table entry size} + PTBR$ → read this from page table, if valid = 1, physical page number + page offset = physical address (of page table entry)

Example translation:

→ what is phys. addr. of virtual addr. 0x5F20? → VPN = 5 → entry 5 in page table indicates VPN 5 is in phys. page 1
 → phys. address = 0x1F20 (1 + offset F20)



Translation Lookaside Buffer: (TLB)

- page table read on every load/store would have severe perf. impact → doubling their delay
- page table accesses have great temporal locality → temporal/spatial locality of data accesses + large page size → many consecutive loads/stores likely to reference same page
- if processor remembers last read page table entry, it can probably reuse this translation without rereading page table
- processor can keep last several page table entries in a small cache called TLB → processor looks aside to find translation in TLB before having to access page table → in real programs, vast majority of accesses hit TLB
- it is a fully associative cache with 16 - 512 entries → each entry contains virtual page number and its corresponding physical page number
- accessed using VPN, returns PPN on a hit, else it goes to page table
- meant to be small enough to be accessed in 1 cycle → even so, TLB hit rate usually, > 99%

Mem. protection:

- in real OS's, several processes run concurrently at any given time → all of these programs are simultaneously present in phys. mem.
- programs should be able to access another program's memory without permission → memory protection
- virtual mem. systems provide mem. protection by giving each program its own virtual address space
- each program can use all the mem. in its virtual address space but only portion of that space is in phys. mem. at any given time
- doesn't have to worry about "avoiding collisions" with other programs, but can only access physical pages mapped in its page table
- sometimes, multiple programs access common data → OS adds control bits to each page table entry to determine who can write to physical pages

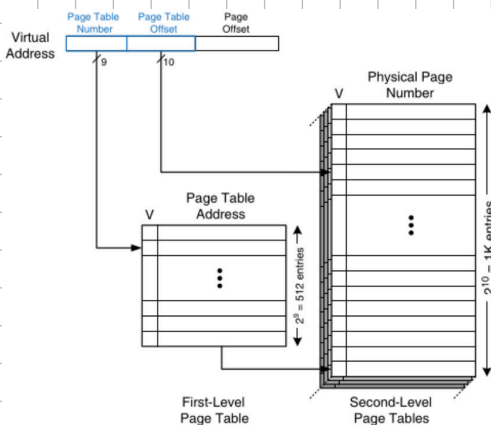
Replacement Policies:

- VMem systems use write-back and approximate LRU replacement policy → write-through impractical because ops. bottlenecked by disk speed
- under write-back, phys. page only written back to disk when evicted from phys. mem.

- writing phys. page back to disk and reloading it with diff. virtual page is called **swapping** → disk in V/Mem system sometimes called **swap space**
- processor swaps out one of LRU phys. pages on a page fault, then replace that with the missing virtual page
- to support this each page table contains **2 additional status bits**: dirty bit and use bit
- **dirty bit = 1** if any store ins. have changed phys. page since it was read from disk → when phys. page swapped out, it only needs to be written back to disk if its dirty bit is 1, else, disk also holds exact copy of page
- **use bit = 1** if phys. page accessed recently → as in caches, exact LRU is impractically complicated → OS approximates LRU by periodically resetting all use bits in page table
- on a page fault, OS finds a page with **U = 0** to swap out of phys. mem. → replaces one of LRU pages

Multilevel Page Tables:

- Page tables can occupy large amount of phys. mem. → previous example would need 2^{19} entries → each entry 4 bytes → page table is 2^{19} bytes = 2MB
- they can be broken up into multiple levels → first-level page table always in phys. mem. → indicates where small **2nd-level page tables** stored in V/Mem
- 2nd-level page tables contain actual translations for a range of virtual pages → if particular range of translations is not actively used, the corresponding 2nd-level page table can be swapped out to hard disk not to waste phys. mem.
- in a 2-level page table, VPN split into page table number and page table offset



→ page table number indexes 1st-level page table in phys. mem.

→ 1st level page table entry gives base address of 2nd-level page table or indicates that it must be fetched from disk when V is 0

→ page table offset indexes 2nd-level page table → remaining 22 bits in example → page offset

→ in our example, 19-bit VPN split into 9-bit page table number and 10-bit page table offset

→ 1st-level page table has $2^9 = 512$ entries → each 2nd-level page table has $2^{10} = 1K$ entries

→ if each 1st/2nd level page table entry is 32 bit (4-byte), hierarchical page table uses $(512 \cdot 4 \text{ bytes}) + 2 \times (1K \times 4 \text{ bytes}) = 10KB$ of phys. mem → fraction of total 2MB page table

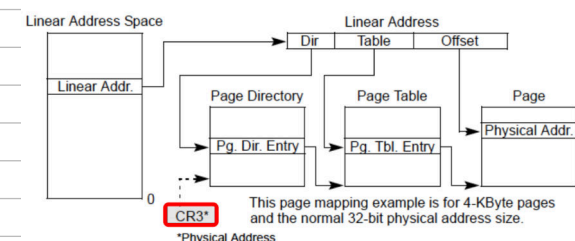
→ **tradeoff**: one more mem. access for translation when TLB misses

→ some x86 stuff building on it (not exam relevant):

Multi-Level Page Tables from x86

Manual

Example from the x86 architecture



CR3: Control Register 3 (or Page Directory Base Register)

x86 Page Tables (I): Small Pages

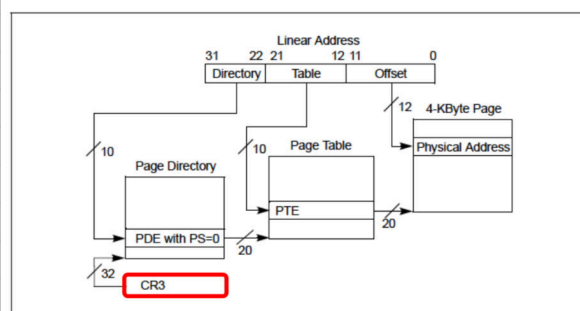


Figure 4-2. Linear-Address Translation to a 4-KByte Page using 32-Bit Paging

x86 Page Tables (II): Large Pages

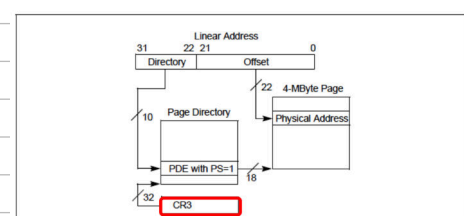


Figure 4-3. Linear-Address Translation to a 4-MByte Page using 32-Bit Paging

Four-level Paging in x86-64

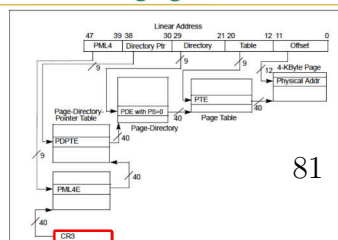
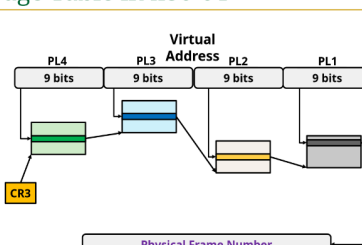
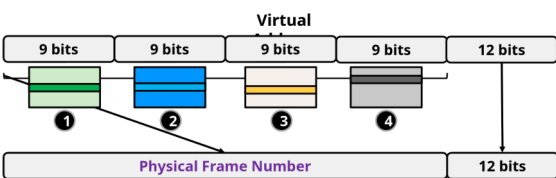


Figure 4-8. Linear-Address Translation to a 4-KByte Page using 4A-32e Paging

Page Table in x86-64



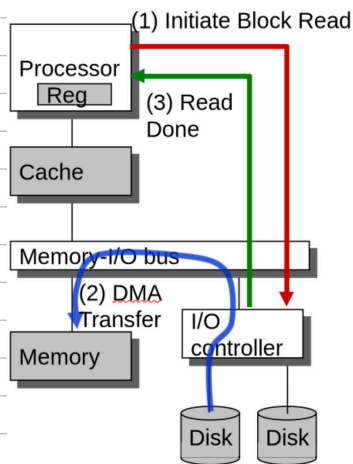
Page Table Walk in x86-64



Four sequential memory accesses during a page table walk in x86-64

Servicing Major Page Fault:

1. Proc. signals I/O controller → read block of length P starting at disk addr. X and store starting at mem. addr. Y
2. disk-to-mem read → Direct Memory Access (DMA) under control of I/O controller
3. controller signals completion via interrupt/ OS resumes suspended process



Privilege Levels in x86

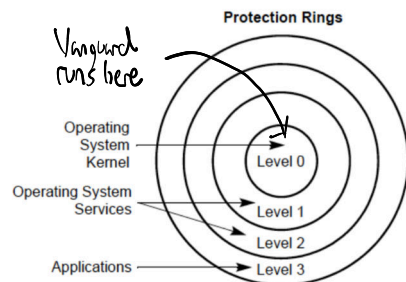


Figure 5-3. Protection Rings

- Four privilege levels in x86 (referred to as rings)

- Ring 0: Highest privilege (operating system) "Supervisor"
- Ring 1: Not widely used
- Ring 2: Not widely used
- Ring 3: Lowest privilege (user applications) "User"

- Supervisor = Kernel (in modern terminology)

→ Was Ring 2 not for drivers? Or is that OS-specific?

x86: A Closer Look at the PDE/PTE

- PDE: Page Directory Entry (32 bits)
- PTE: Page Table Entry (32 bits)

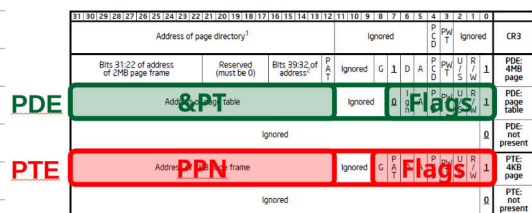


Figure 4-4. Formats of CR3 and Paging-Structure Entries with 32-Bit Paging

Protection: PDE + PTE = ???

Page-Directory Entry	Access Type	Page-Table Entry	Access Type	Combined Effect	Access Type
User	Read Only	User	Read Only	User	Read Only
User	Read Only	User	Read-Write	User	Read-Write
User	Read-Write	User	Read Only	User	Read-Write
User	Read-Write	User	Read-Write	User	Read-Write
Supervisor	Read Only	Supervisor	Read Only	Supervisor	Read-Write*
Supervisor	Read Only	Supervisor	Read-Write	Supervisor	Read-Write*
Supervisor	Read-Write	Supervisor	Read Only	Supervisor	Read-Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read-Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read-Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read-Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read-Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read-Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read-Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read-Write*

NOTE
* If CR0.WP = 1, access type is determined by the R/W flags of the page-directory and page-table entries. If CR0.WP = 0, supervisor privilege permits read-write access.

x86 Page Directory Entry (PDE)

Protection: PDE's Flags

- Protects all 1024 pages in a page table

Bit Position(s)	Contents
0 (P)	Present; must be 1 to reference a page table
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-MByte region controlled by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-MByte region controlled by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether this entry has been used for linear-address translation (see Section 4.8)
6	Ignored
7 (PS)	If CR4.PSE = 1, must be 0 (otherwise, this entry maps a 4-MByte page; see Table 4-4); otherwise, ignored

x86 Page Table Entries

Figure 4-4 gives a summary of the formats of CR3 and the paging-structure entries with 32-bit paging. For the paging structure entries, it identifies separately the format of entries that map pages, those that reference other paging structures, and those that do neither because they are "not present"; bit 0 (P) and bit 7 (PS) are highlighted because they determine how such an entry is used.

Bit Position(s)	Contents
0 (P)	Present; must be 1 to map a 4-KByte page
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-KByte page referenced by this entry (depends on CPL and CR0.WP; see Section 4.6)
2 (U/S)	User/supervisor; if 0, accesses with CPL=3 are not allowed to the 4-KByte page referenced by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether software has accessed the 4-KByte page referenced by this entry (see Section 4.8)
6 (D)	Dirty; indicates whether software has written to the 4-KByte page referenced by this entry (see Section 4.8)
7 (PAT)	If the PAT is supported, indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2); otherwise, reserved (must be 0)
8 (G)	Global; if CR4.PGE = 1, determines whether the translation is global (see Section 4.10); ignored otherwise
11:9	Ignored
31:12	Physical address of the 4-KByte page referenced by this entry

Protection: PTE's Flags

- Protects one page at a time

Bit Position(s)	Contents
0 (P)	Present; must be 1 to map a 4-KByte page
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-KByte page referenced by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-KByte page referenced by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether software has accessed the 4-KByte page referenced by this entry (see Section 4.8)
6 (D)	Dirty; indicates whether software has written to the 4-KByte page referenced by this entry (see Section 4.8)
7 (PAT)	If the PAT is supported, indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2); otherwise, reserved (must be 0)
8 (G)	Global; if CR4.PGE = 1, determines whether the translation is global (see Section 4.10); ignored otherwise
11:9	Ignored
31:12	Physical address of the 4-KByte page referenced by this entry

x86 PTE (4KB page)

Table 4-6. Format of a 32-Bit Page-Table Entry that Maps a 4-KByte Page

Bit Position(s)	Contents
0 (P)	Present; must be 1 to map a 4-KByte page
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-KByte page referenced by this entry (depends on CPL and CR0.WP; see Section 4.6)
2 (U/S)	User/supervisor; if 0, accesses with CPL=3 are not allowed to the 4-KByte page referenced by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether software has accessed the 4-KByte page referenced by this entry (see Section 4.8)
6 (D)	Dirty; indicates whether software has written to the 4-KByte page referenced by this entry (see Section 4.8)
7 (PAT)	If the PAT is supported, indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2); otherwise, reserved (must be 0)
8 (G)	Global; if CR4.PGE = 1, determines whether the translation is global (see Section 4.10); ignored otherwise
11:9	Ignored
31:12	Physical address of the 4-KByte page referenced by this entry

Table 4-5. Format of a 32-Bit Page-Directory Entry that References a Page Table

Bit Position(s)	Contents
0 (P)	Present; must be 1 to reference a page table
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-MByte region controlled by this entry (depends on CPL and CR0.WP; see Section 4.6)
2 (U/S)	User/supervisor; if 0, accesses with CPL=3 are not allowed to the 4-MByte region controlled by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the page table referenced by this entry (see Section 4.9)
5 (A)	Accessed; indicates whether this entry has been used for linear-address translation (see Section 4.8)

X86-64 Page Table Entry Structure

X86-64 Page Table: Accessing 4KB pages

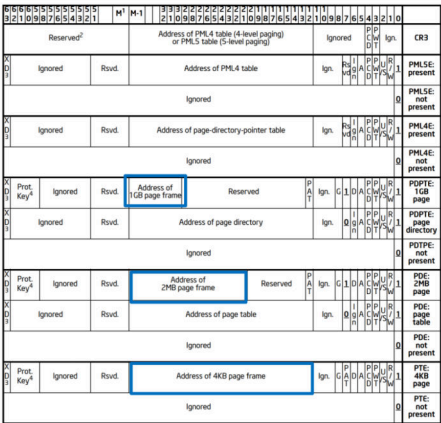


Figure 4-11. Format of CR3 and PML4 table entries with 4-level paging and 5-level paging

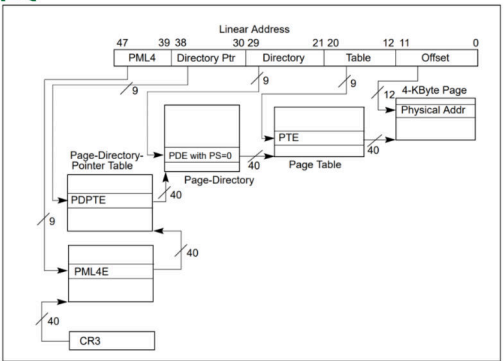


Figure 4-8. Linear-Address Translation to a 4-KByte Page using 4-Level Paging

X86-64 Page Table: Accessing 2MB pag

X86-64 Page Table: Accessing 1GB pa

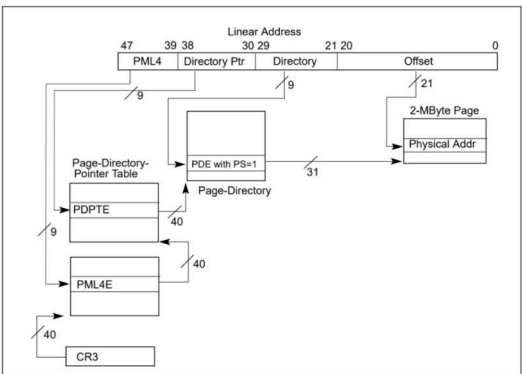


Figure 4-9. Linear-Address Translation to a 2-MByte Page using 4-Level Paging

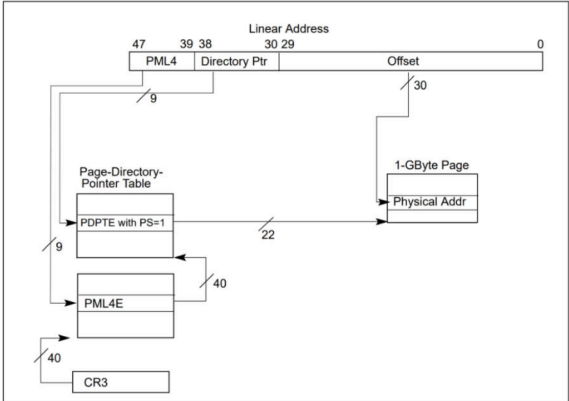
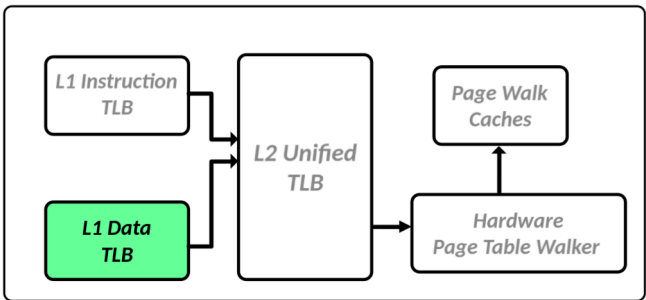


Figure 4-10. Linear-Address Translation to a 1-GByte Page using 4-Level Paging

Intel Skylake: L1 Data TLB



8 Tomasulo's Algorithm [60 points]

Consider an in-order fetch, out-of-order dispatch, and in-order retirement execution engine that employs Tomasulo's algorithm. This engine has the following characteristics:

- The engine has four main pipeline stages: Fetch (F), Decode (D), Execute (E), and Write-back (W).
- The engine can fetch one instruction per cycle, decode one instruction per cycle, and write back the result of one instruction per cycle.
- The engine has two execution units: 1) an adder to execute ADD instructions and 2) a multiplier to execute MUL instructions.
- The execution units are fully pipelined. The adder has two stages (E1-E2), and the multiplier has four stages (E1-E2-E3-E4). Execution of each stage takes one cycle.
- The adder has a two-entry reservation station, and the multiplier has a three-entry reservation station.
- An instruction always allocates the first available entry of the reservation station (in top-to-bottom order) of the corresponding execution unit.
- Full data forwarding is available, i.e., during the last cycle of the E stage, the tags and data are broadcast to the reservation station and the Register Alias Table (RAT). For example, an ADD instruction updates the reservation station entries of the dependent instructions in the E2 stage. So, the updated value can be read from the reservation station entry in the next cycle. Therefore, a dependent instruction can potentially begin its execution in the next cycle (after E2).
- The multiplier and adder have separate output data buses, which allow both the adder and the multiplier to update the reservation station and the RAT in the same cycle.
- An instruction continues to occupy a reservation station slot until it finishes the Write-back (W) stage. The reservation station entry is deallocated after the Write-back (W) stage.

8.1 Problem Definition

The processor is to fetch and execute five instructions. Assume the reservation stations (RS) are all initially empty, and the initial state of the register alias table (RAT) is given below in Figure (a). Instructions are fetched, decoded, and executed as discussed in class. At some point during the execution of the five instructions, a snapshot of the state of the RS and the RAT is taken. Figures (b) and (c) show the state of the RS and the RAT at the snapshot time. A dash (-) indicates that a value has been cleared. A question mark (?) indicates that a value is unknown to you.

Reg	Valid	Tag	Value
R0	1	-	256
R1	1	-	28
R2	1	-	1
R3	1	-	3
R4	1	-	30
R5	1	-	5
R6	1	-	23
R7	1	-	20
R8	1	-	61
R9	1	-	4

(a) Initial state of the RAT

Reg	Valid	Tag	Value
R0	1	7	256
R1	1	7	28
R2	1	7	1
R3	1	7	50
R4	1	7	30
R5	0	X	7
R6	1	7	23
R7	0	Y	7
R8	0	Z	7
R9	0	T	7

(b) State of the RAT at the snapshot time

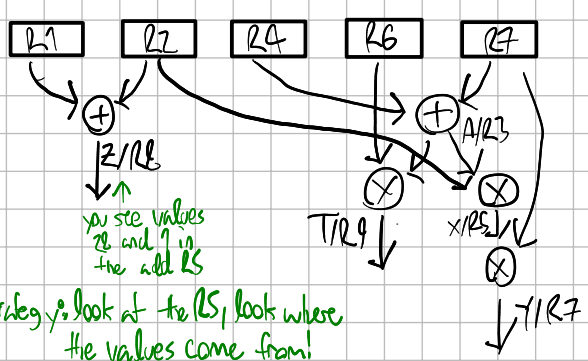
ID	V	Tag	Value	V	Tag	Value
-	-	-	-	-	-	-
Z	1	7	28	1	7	1

(c) State of the RS at the snapshot time

→ OK that's enough, this is ridiculous
→ and with that I'll conclude the theory part of the sheet and tack on some exam task walkthroughs for you
→ the following are from FS22: (let's only do tricky ones)

Tomasulo:

8.2 Questions
8.2.1 Dataflow Graph [40 points]
Based on the information provided above, identify the instructions and provide the dataflow graph below for the instructions that have been fetched. Please appropriately connect the nodes using edges and specify the direction of each edge. Label each edge with the destination architectural register and the corresponding Tag.



8.2.2 Program Instructions [20 points]
Fill in the blanks below with the five-instruction sequence in program order. There can be more than one correct ordering. Please provide only one correct ordering. When referring to registers, please use their architectural names (R0 through R9). Place the register with the smaller architectural name on the left source register box.
For example, ADD R8 ← R1, R5.

1. ADD R3 ← R4, R7 (first because cleared already)
2. ADD R8 ← R1, R2 (second because the renamed register doesn't feed into anything)
3. MUL R5 ← R2, R3
4. MUL R7 ← R5, R7

5. MUL R4 <= R3, R6
 ↳ order taken from mult. RS

→ I'll just say the same advice as for the dataflow graph, just here you have to pay attention to what already cleared!

GPU/SIMD:

We define the *SIMD utilization* of a program that runs on a GPU as the fraction of SIMD lanes that are kept busy with *active threads* during the run of a program. As we saw in lecture and practice exercises, the SIMD utilization of a program is computed across the *complete run* of the program.

The following code segment is run on a GPU. A warp in the GPU consists of 32 threads, and there are 32 SIMD lanes in the GPU. Each thread executes a **single iteration** of the shown loop. Assume that the data values of the arrays A, B and C are already in vector registers so there are no loads and stores in this program. Both B and C are arrays of integers and each integer in these arrays has an absolute value of less than 10 (i.e., $|B[i]| < 10$ and $|C[i]| < 10$, for all i).

```
for (i = 0; i < 1024; i++) {
    A[i] = B[i] * C[i]; // instruction 1
    if (/* Condition */) { // instruction 2
        // instruction 3
        // instruction 4
        .
        .
        // instruction k + 2
    }
    C[i] = C[i] - 1; // instruction k + 3
}
```

(c) [20 points] Assume that the condition for the if statement is $(i \% 16 == 0 \wedge i < 512)$. What is the number of instructions (k) in the body of the conditional block given a SIMD utilization of $\frac{9}{16}$? Assume that there are **no** control flow instructions in the body of the if statement. Show your work.

(a) [5 points] How many warps does it take to execute this program? Show your work.

1024 iterations: # warps = $\frac{1024}{32} = 32$

Show Answers

(b) [20 points] Assume that the condition for the if statement is $(i \% 16 == 0)$. What is the number of instructions (k) in the body of the conditional block given a SIMD utilization of $\frac{11}{32}$? Assume that there are **no** control flow instructions in the body of the if statement. Show your work.

→ 32 lines always executed, if statement b 1 has k lines
 → warp has 32 lines, $i \% 16 == 0 \Rightarrow$ if statement body executes twice per warp
 $\Rightarrow$ all $k+3$ ins. loaded for every warp $\Rightarrow$ efficiency same per warp

$$\Rightarrow \text{efficiency} = \frac{32 \cdot 3 + 2k}{32(k+3)} = \frac{11}{32} \Rightarrow \frac{32 \cdot 3 + 2k}{32(k+3)} = \frac{11(k+3)}{32(k+3)} \Rightarrow 96 + 2k = 11(k+3)$$

$$\Rightarrow 96 + 2k = 11(k+3) \Rightarrow 96 + 2k = 33 + 11k \Rightarrow 9k = 63 \Rightarrow k = 7$$

→ crazy how this is just algebra + interpretation, i.e. that's engineer math

→ usually, the fraction for efficiency would be $\frac{32 \cdot (32 \cdot 3 + 2k)}{32 \cdot 32(k+3)}$ because each set of ins. is reloaded and executed in each warp → number of ins. loaded & executed constant for every warp $\Rightarrow$ 32s cancel each other out

→ this is what we just discussed on the right:

only the first 16 warps even execute the statement

$\Rightarrow$ total # of loaded ins. = $16 \cdot 32 \cdot (k+3) + 16 \cdot 32 \cdot 3$

$\Rightarrow$ total # of executed ins. = $16(32 \cdot 3 + 2k) + 16 \cdot 32 \cdot 3$

$\Rightarrow$ efficiency fraction: $\frac{16(32 \cdot 3 + 2k) + 16 \cdot 32 \cdot 3}{16 \cdot 32(k+3) + 16 \cdot 32 \cdot 3}$

$$\Rightarrow \frac{16(32 \cdot 3 + 2k) + 16 \cdot 32 \cdot 3}{16 \cdot 32(k+3) + 16 \cdot 32 \cdot 3} = \frac{5}{2} \Rightarrow \frac{16(32 \cdot 3 + 2k) + 16 \cdot 32 \cdot 3}{16 \cdot 32(k+3+3)} = \frac{5}{2} \Rightarrow \frac{10 \cdot 32(k+6)}{16 \cdot 32(k+3+3)} \Rightarrow 16(32 \cdot 3 + 2k) + 16 \cdot 32 \cdot 3 = 10 \cdot 32(k+6)$$

$$\Rightarrow 16(32 \cdot 3 + 2k) + 16 \cdot 32 \cdot 3 = 20 \cdot 16(k+6) \Rightarrow 32 \cdot 3 + 2k + 32 \cdot 3 = 20(k+6) \Rightarrow 96 + 2k + 96 = 20k + 120 \Rightarrow 192 = 18k + 120$$

$$\Rightarrow 72 = 18k \Rightarrow k = 4$$

Branch Prediction:

A processor implements an *in-order* pipeline with multiple stages. Each stage completes in a single cycle. The pipeline stalls after fetching a *conditional branch instruction* and resumes execution once the condition of the branch is evaluated. There is no other case in which the pipeline stalls.

10.1 Part I: Microbenchmarking [30 points]

You create a microbenchmark as follows to reverse-engineer the pipeline characteristics:

```
LOOP1:
    SUB R1, R1, #1 // R1 = R1 - 1
    BGT R1, LOOP1 // Branch to LOOP1 if R1 > 0

LOOP2:
    B LOOP2 // Branch to LOOP2
    // Repeats until program is killed
```

The microbenchmark takes one input value R1 and runs until it is killed (e.g., via an external interrupt).

You carefully run the microbenchmark using different input values (R1) as summarized in Table 2. You terminate the microbenchmark using an external interrupt such that each run is guaranteed to execute exactly 50 dynamic instructions (i.e., the instructions actually executed by the processor, in contrast to static instructions, which is the number of instructions the microbenchmark has).

Initial R1 Value	Number of Cycles Taken
2	71
4	83
8	107
16	155

Table 2: Microbenchmark results.

Using this information, you need to determine the following two system characteristics. *Clearly show all work to receive full points!*

- How many stages are in the pipeline?
- For how many cycles does a conditional branch instruction cause a stall?

Code Reverse Engineering:

12 BONUS: Cache Reverse Engineering [70 points]

You are trying to reverse-engineer the cache associativity in a newly-released system. You already know that the cache has a FIFO replacement policy and 8 blocks with a block size of 4 B. Starting with an empty cache, an application accesses five byte addresses in the following order

2 → 9 → 16 → 25 → 33

Assume you can access three addresses after the above sequence and observe the cache hit rate across these three accesses.

- [30 points] Which three addresses should you access in order to identify the set-associativity of the cache (1-, 2-, 4- or 8-way)? There may be multiple solutions; please give the lowest possible addresses that can enable the identification of the set-associativity. Please explain every step in detail to get full points.

→ I mean one can naively guess that the answer for 2 is 6, but let's do it the official way
 ↳ Cycles taken = pipeline dynamic ins. ↳ # cycles stalled per conditional branch

→ formula for cycles taken: $C = P + I - 1 + 8D$

→ we know that $I=50$ from description $\Rightarrow I-1=49$ ↳ # conditional branches taken

→ look at code: BGT branch taken as many times as R1 value

$$\Rightarrow \text{for R1 value 2: } 71 = P + 49 + 20(I) \Rightarrow 20 = 12 \Rightarrow D = 6 (= \text{answer to 2.})$$

$$\text{for R1 value 4: } 83 = P + 49 + 4D$$

$$\text{in I: } 71 = P + 49 + 12 \Rightarrow P = 10 (= \text{answer to 1.})$$

Master Solution:

→ there are four possible set/way configurations:

1 set, 8 ways: 0 4 8 12 16 20 24 28
 2 9 16 25 33 - - -

2 sets, 4 ways: 0 4 8 12 16 20 24 28
 33 9 16 25 - - -
 4 sets, 2 ways: 0 4 8 12 16 20 24 28
 9 25 - - -
 8 sets, 1 way: 33 16 - - -
 9 25 - - -
 25 24 28 - - -

33 9/32...
 - 4
 9 8
 - 12
 16 16
 - 20
 25 24 28
 - - -

→ here, all 4 cache associativities have 100% miss rate (started cold) → to differentiate the 4 cases, we need to induce different hit/miss counts in all the associativities: 1 cache needs 3 hits, another needs 2, another 1, another 0

→ Key tip: (I was so confused here) example: if the cache contains 2, an access to 0 is still a hit, because both are in block 0-3
→ 0, 1, 2, 3 are all the same cache block!

→ you will quickly find that the accesses $[0-3] \rightarrow [8-11] \rightarrow [16-19]$ and $[0-3] \rightarrow [16-19] \rightarrow [32-35]$ are the only valid solutions, only the former is correct as it is lower

very nice corollary:

2. [20 points] What is the associativity of the cache if the cache hit rate observed over the 3 extra addresses accessed in Part (1) were:

Hit rate	Associativity
0	4-way, 2-set
1/3	2-way, 4-set
2/3	1-way, 8-set
1	2-way, 1-set

3. [20 points] When you accessed the three addresses you determined in Part (1), you observed a 100% hit rate across these three accesses. Now, your friend asks you to access four more addresses in the following order:

$32 \rightarrow 0 \rightarrow 8 \rightarrow 28$

Which of the above four addresses would result in a cache miss?

⇒ we are working with the 8-way cache ⇒ 28 would result in a miss
→ exercise for you: prove it by drawing it up!

→ Final note: don't underestimate DCA because there are many easy questions (I didn't show you those (see!)), you will see the design is "mean" in that if you don't get the first part of a question right, you often won't get the other parts

→ you have now reached the end of the longest handwritten document you've ever skimmed! Thank you for following along with my messy handwriting / making this was so intense I had to have Churchill speeches as motivation in the background!